

Q.1
 → Return index first occurrence
 expectation → $f(arr, 0, 2)$
 faith → $f(arr, index+1, 2)$
 small → $arr.length-1$

psum() {

int num = 2, arr = {5, 2, 3, 2, 4}

→ int index = f(arr, 0, num);

if (index == -1) { number not found };

else { in smp(index) } ← $\frac{1}{2}$

int f(int [] arr, int index, int num) {

if (index == arr.length) {

return -1;

if (index == arr[index] == num) {

return index;

return f(arr, index+1, num); ← $\frac{1}{2}$

Return index last occurrence

expectation → $f(arr, 4, 2)$

faith → $f(arr, index-1, 2)$

small → -1

psum() {

int [] arr = {5, 2, 3, 2, 4}

int marks = f(arr, 4, 2) arr.length-1, 2)

Q.4

int [] arr = [1, 2, 3, 4]

HashMap<Integer, Integer> hm = new HashMap<>();

for (int i = 0; i < arr.length - 1; i++) {

Q.4

psumc {

int [] arr = [1, 2, 3, 4]

[1, 0, 4, 9]

boolean out = f(arr, 0);

Sop(out);

}

a boolean of (int [] arr, int index) {

if (arr[index] < arr[index + 1]) {

return true;

}

if

if (arr[index] > arr[index + 1]) {

return false;

}

if (arr[index] ==

if (index == arr.length - 1) {

return true;

}

return f(arr, index + 1);

}

all elements are even.

Q.5 → Psvmcs {
int arr = [2, 4, 6];
boolean even = f(arr, 0);
Sop(even);
}

```
int boolean f(int arr, int index) {  
    if (index == arr.length - 1) {  
        if (arr[index] % 2 == 0) {  
            return false;  
        }  
        if (index == arr.length - 1) {  
            return true;  
        }  
        return f(arr, index + 1);  
    }  
}
```

Q.6. Psvmcs {
int arr = [1, 2, -3, 4];
boolean negative = f(arr, 0);
Sop(negative);
}

```
boolean f(int arr, int index) {  
    if (arr[index] % 2 != 0) {  
        return true;  
    }  
    if (arr[index] == arr.length - 1) {  
        return false;  
    }  
    return f(arr, index + 1);  
}
```


Q.7 psuvc {
 int arr = {3, 3, 1, 7}, max = arr[0]
 int max = f(arr, 0, max)
 if sop(max):
 }
 int f(int arr, int index, int max) {
 int max = arr[index];
 if (arr[index] > max) {
 max = arr[index];
 }
 if (arr index == arr length - 1) {
 return max
 }
 return (arr, index+1, max);
 }

Q.8 psuvc {
 int arr = {3, 9, 1, 7}, min = arr[0]
 int min = f(arr, 0, min);
 sop(min);
 }
 int f(int arr, int index, int min) {
 if (arr length [index] < min) {
 min = arr[index];
 }
 if (arr length - 1 == index) {
 return min;
 }
 return f(arr, index+1, min);
 }

Q. 9] psumc {
 int arr = {10, 20, 30, 40, 50}; int Add = 0;
 int sum = f(arr, 0, Add);
 sop(sum);
 }
 int f(int arr, int index, int Add) {
 if (index % 2 != 0) {
 Add sum += arr[index];
 }
 if (index == arr.length - 1) {
 return Add;
 }
 return f(arr, index, Add);
 }

$$5 + 1 = 6/2 = 3 \quad 6/2 + 1 = 2 + 3$$

Q. 10. psumc {
 int arr = [1, 2, 3, 2, 1];
 int boolean out = f(arr, 0);
 sop(out);
 }

$$5/2 + 1 = 2 + 1 = 3 \quad 5 - 1/2 = 4/2$$

boolean f(int arr, int index) {
 if (arr[index] == arr[arr.length - 1 - index]) {
 if (arr[index] != arr[arr.length - 1 - index]) {
 return false;
 }
 if (index == arr.length - 1/2) {
 return true;
 }
 return f(arr, index + 1);
 }

Q11

psvm {

int arr = {10, 5, 8, 20, 15}

int max = ~~arr~~ arr[0]; int secondmax = arr[0];

int max2 = f(arr, 0, max, secondmax);

Sop(max2);

}

int f(int arr[], int index, int max, int sm) {

if (arr[index] > max) {

max = ~~int~~ temp = max;

secondmax = max;

max = arr[index]

}

else if (arr[index] < max && arr[index] > secondmax) {

secondmax = arr[index];

}

if (index == arr.length - 1) {

return secondmax;

}

return f(arr, index+1, max, sm);

}

Q12

psvm {

int arr = {1, 2, 3, 3};

int

i

int boolean b = f(arr, 0

Sop(b);

}

boolean f(int arr[], int index) {

if (index == arr.length) {

return true;

}

if (arr[index] > arr[index+1])

return f(arr, index+1)

}

Q.13 psumc {
 int arr = [1, 5, 3, 7], int x = 3
 int count = 0;
 int total count = f(arr, 0, x, count);
 sip(TC);
 3
 int f(int arr, int index, int x, int count) {
 if (index == arr.length) {
 return count;
 }
 if (arr[index] > x) {
 count++;
 }
 return f(arr, index+1, x, count);
 }

Q.14. psumc {
 int arr = [1, 2, 3, 4], int t = 3.
 int target = f(arr, 1, 0, t);
 sup(target);
 3
 int f(int arr, int index, int t) {
 if (index == arr.length) {
 return false;
 }
 if (arr[index] == t) {
 return true;
 }
 return f(arr, index+1, t);
 }

Q-15

```

psumc1 {
    int arr[1, -2, 9, -4, 5], int Add;
    int sum = 0;
    sum = f(arr, 0, sum);
    sop(sum);
}

int f(int arr, int index, int sum) {
    if (index == arr.length) {
        return sum;
    }
    if (arr[index] < 0) {
        sum = sum + arr[index];
    }
    return f(arr, index + 1, sum);
}
    
```

Q-16

```

psumc2 {
    int arr = [5, 2, 8, 1, 3], int min = arr[0], int
    int index = f(arr, 0, min, 0);
    sop(index);
}

int f(int arr, int index, int min, int mi) {
    if (arr.length == index) {
        return index, mi;
    }
    if (arr[index] < min) {
        min = arr[index];
    }
    return f(arr, index + 1, min, mi);
}
    
```

```

Q.17. psolve {
    int arr[2, 3, 5, 6, 9], int k=2
    int count = 0;
    count = f(arr, 0, k, count);
    sup(count);
}
int f(int arr, int index, int k, int count) {
    if (arr[index] == arr.length) {
        return count;
    }
    if (arr[index] % k == 0) {
        count++;
    }
    return (arr, index+1, k, count);
}

```

```

Q.18. psolve {
    int arr[1, 2, 3, 4];
    bool unique = f(arr, 0,);
    sup(unique);
}
bool f(int arr, int index, i)
    for (i=0; i < arr.length-1; i++) {
        for (int j=i+1; j < arr.length-1; j++) {
            if (arr[i] == arr[j]) {
                return false;
            }
        }
    }
    return true;
}

```


Q.19. summe 13
int arr = [12, 13, 5];

int digit = 0;

digit = f(arr, 0, digit);

Sop(digit);

int f(int arr, int index, int digit) {

if (index == arr.length) {

return digit;

int n = arr[index];

while (arr[index] > 0) {

int rem = arr[index] % 10; n = n / 10;

digit = digit * 10 + rem;

arr[index] = n;

n = n / 10;

}

return f(arr, index + 1, digit);

}

Q.20 summe 13

int arr = [9, 7, 5, 2];

boolean out = true; f(arr, 0);

Sop(out);

3

boolean f(int arr, int index) {

if (index == arr.length - 1) {

return true;

3

if (arr[index] <= arr[index + 1]) {

return false;

return f(arr, index + 1);